



**Penguins**



# Preliminary: Reinforcement Learning

- *Reinforcement learning* is a type of machine learning.
- It allows an *agent* to learn by interacting with an environment.
- The agent receives rewards for *positive* actions and punishments for *negative* ones.
- The goal is to **maximize** the cumulative **reward** over time.



# PPO vs MORL

- **Simple PPO:** Agent learns a single task within an environment.
- **Multi-Objective RL (MORL):** Agent learns multiple objectives/tasks.

## Our Implementation

- **Time constraint** - Complete task quickly.
- **Movement conservation** - Minimize energy use.
- **Feeding baby penguin** - Primary goal.





# PPO vs MORL

## Analysis

- **Observing Penguin Agent Actions:** Compare PPO and MORL agents' actions at each timestep.
- **Essence of Research:** Analyze why the agents take specific actions.
- **PPO vs MORL:** Does MORL optimize for time and energy while feeding the baby penguin?
- **Potential PPO Issues:** Does PPO lead to wandering or excessive time-wasting?



# PPO Configuration Details

- **trainer\_type:** ppo
- **batch\_size:** 128
- Determines the number of experiences (state, action, reward, next state) that are grouped together for a single update of the neural network during training.
- **buffer\_size:** 2048
- Defines the total number of experiences collected from the environment before a policy update is performed.
- **learning\_rate:** 0.0003
- It determines how much the policy adjusts its parameters based on the gradients calculated during training.
- **num\_epoch:** 3
- Specifies the number of passes the PPO algorithm makes through the collected experience buffer during one policy update.
- **max\_steps:** 1000000
- Determines the maximum number of steps of interaction (or frames of simulation) that the agent takes with the environment before training is terminated.

# Experiment Overview & Setup



## Goal

- Train a "Penguin" agent using Proximal Policy Optimization (PPO).
- **Framework:** ML-Agents v1.1.0, PyTorch v2.2.2+cu121

## Configuration

- Both runs used the same training configuration file (config/ppo/Penguin.yaml).

## Agent Implementation

- **Run Penguin\_01:** PenguinSimpleAgent.cs (Basic reward structure)
- **Run Penguin\_06:** PenguinAgent.cs (Reward shaping / MORL-inspired structure)
- **Total Steps:** Both runs completed ~1 million steps.

# Learning Dynamics: Agent Behavior

## Metric 1: Mean Episode Length (Steps per episode, late training)

- **Penguin\_01**: ~253 steps
- **Penguin\_06**: ~248 steps
- **Observation**: Higher-performing agent (06) finished episodes slightly faster.
- This aligns with its reward structure penalizing movement, encouraging efficiency.



# Learning Dynamics: Agent Behavior

## Metric 2: Policy Entropy (Measure of policy randomness, late training)

- **Penguin\_01:** ~0.562
- **Penguin\_06:** ~0.549
- **Observation:** Penguin\_06 had slightly lower entropy, suggesting its learned policy might be slightly more deterministic or "confident", likely due to the clearer goal and efficiency incentives.





# Learning Signals - Loss Functions

**Metric 1: Policy Loss (How much the policy action probabilities are updated)**

- **Penguin\_01:** ~0.0694
- **Penguin\_06:** ~0.0703
- **Observation:** Very similar values. The core PPO update mechanism behaved similarly in magnitude.



# Learning Signals - Loss Functions



## Metric 2: Value Loss (Error in predicting future rewards)

- **Penguin\_01:**  $\sim 0.1308$
- **Penguin\_06:**  $\sim 0.4058$
- **Observation:** Penguin\_06 had a significantly higher average value loss.
- This doesn't mean worse learning. Predicting the state values for the much higher-reward and potentially more complex trajectory learned by Penguin\_06 was likely harder for the value network.

# Penguin\_01: The "Simple" Agent



**Objective: Feed the baby penguin.**

- **Reward Logic:**
- +1f reward for eating a fish.
- +1f reward for successfully feeding the baby.
- Small negative reward ( $-1f / \text{MaxStep}$ ) per step.

# Penguin\_06: The "MORL-Inspired" Agent



**Objective: Feed the baby penguin efficiently.**

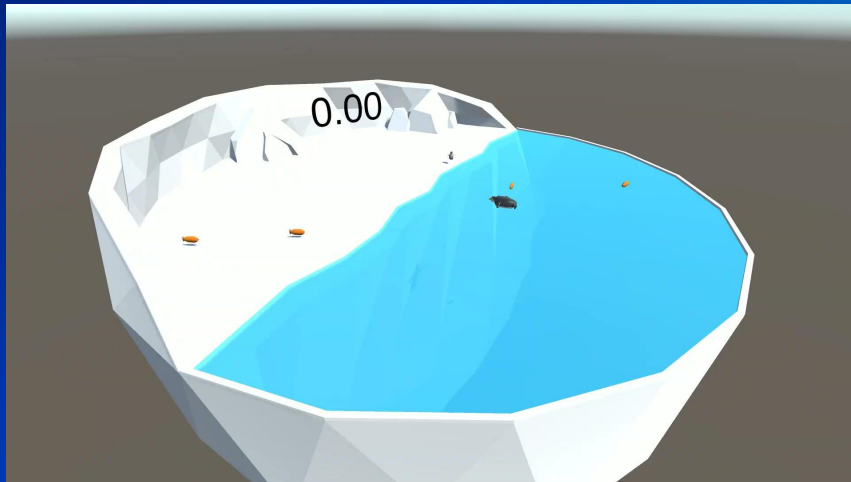
- **Reward Logic:**
- +1f reward for eating a fish.
- +3f reward for feeding the baby (stronger goal incentive).
- Small negative reward ( $-1f / \text{MaxStep}$ ) per step.
- $-0.0002f$  penalty for moving forward.
- $-0.0002f$  penalty for turning.



# Video Comparison



Penguin\_01



Penguin\_06



# Conclusion & Discussion



01 **Reward Shaping is Crucial:** Agent implementation impacts outcomes more than hyperparameter tuning.

02 **Complexity vs. Performance:** Sophisticated reward shaping yields better policies but may need more tuning/training.

03 **Clear Goals Drive Learning:** Stronger primary objective reward led to significantly better results.

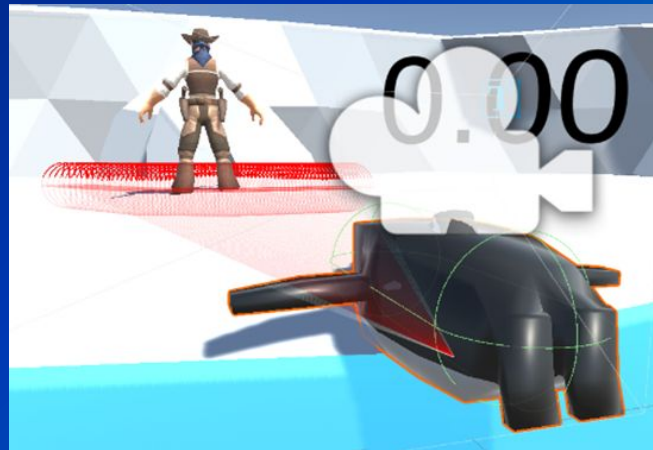
04 **Incentivize Efficiency:** Small penalties for actions effectively guide efficient solutions.

# Visual perception



01

3D Lidar



02

Camera



# Visual perception



- The image data generated by the camera will be used as the main observation input for the intelligent body to support subsequent deep visual perception tasks;
- Compared to simple distance sensors (e.g., laser or radar), image inputs can provide richer semantic information (e.g., recognizing the location and type of fish/penguin).

